Lab: Agentic RAG Uygulaması (LangChain · LangGraph · Qdrant) — Katılımcı Rehberi

Seviye	Orta
Araçlar	Python 3.12, uv, Git, Docker
Çıktı	10 görev, tüm testler yeşil

1. Amaç

Bu lab sonunda şunları yapabilirsin:

- PDF'ten chunk üretip hibrit (dense + BM25) arama kurmak
- Arama sonuçlarını LLM ile yeniden sıralamak
- LangGraph ile düğüm, router ve graph yazmak
- LLM'e araç verip agent kurmak

Uygulamanın geri kalanı hazırdır; sen yalnızca 10 görevi yazarsın.

2. Ön koşullar

- Temel Python bilgisi
- Git ve terminal kullanımı
- Gemini API anahtarı

3. Kurulum

1. Repoyu klonla ve `student` branch'ine geç:

```
git clone https://github.com/KaanAkkok/Langchain_RAG_Playground.git
cd Langchain_RAG_Playground
git checkout student
```

2. Kendi çalışma branch'ini aç:

```
git checkout -b ad-soyad
```

3. Bağımlılıkları kur:

```
uv sync
```

4. Ortam dosyasını oluştur ve `GEMINI_API_KEY` değerini gir:

```
cp .env.example .env          # PowerShell: Copy-Item .env.example .env
```

5. Testleri çalıştır:

```
uv run pytest tests/exercises -q
```

Beklenen: Çoğu test kırmızıdır; bu normaldir. Görevleri bitirdikçe yeşile döner.

4. Çalışma düzeni

Her görevde aynı adımları izle:

1. Görevde belirtilen dosyayı ve fonksiyonu aç.
2. `raise NotImplementedError(...)` satırını sil.
3. Kodunu yaz.
4. İlgili testi çalıştır: `uv run pytest tests/exercises -k gorevN -v`

Notlar:

- Testler API anahtarı, Qdrant ve Docker gerektirmez.
- Test hata mesajları Türkçedir ve neyin eksik olduğunu söyler.
- Uygulama en sonda Docker ile çalıştırılır (Bölüm 5).

5. Uygulamanın akışı

Bir kullanıcı mesajı şu adımlardan geçer:

1. **Input guardrail:** Prompt injection reddedilir, kişisel veri maskelenir.
2. **Intent:** Mesajın niyeti ve bağımsız arama sorgusu (`resolved_query`) belirlenir.
3. **Memory recall:** Kullanıcı hakkındaki eski notlar getirilir.
4. **Retrieval + Reranker:** Qdrant'ta hibrit arama yapılır, adaylar LLM ile puanlanır.
5. **Assistant:** Kaynaklara dayanarak `[1]`, `[2]` atıflı cevap üretilir.
6. **Judge:** Cevap kaynağa sadık mı diye denetlenir; kalırsa asistan yeniden yazar.
7. **Output guardrail:** Cevaptaki kişisel veri maskelenir.
8. **Memory extract:** Konuşmadan kalıcı notlar çıkarılır.

Belgeler önceden **ingestion** ile işlenir: PDF → Markdown → chunk → embedding → Qdrant.

Akış iki şekilde kurulur:

- **Workflow graph:** Akış sabittir; bilgi sorusunda 4. adım her zaman çalışır.
- **Agentic graph:** 4. adım ayrı düğüm değildir; asistan bir agent'tır ve `search_documents` aracını ne zaman çağıracağına kendisi karar verir.

Bölüm 1 — RAG Temel Yapıları

Ön bilgi

- **Chunking:** Belge anlamlı parçalara bölünür: başlığa göre → anlam kopuşuna göre → boyuta göre. Her chunk sayfa ve başlık bilgisi taşır; atıflar buradan gelir.
- **Embedding:** Metni, anlamını temsil eden bir vektöre çevirir. Anlamca yakın metinlerin vektörleri de yakındır.
- **Hibrit arama:** Dense vektör anlamı yakalar ("otomobil" ≈ "araba"); sparse (BM25) tam ifadeyi yakalar ("Madde 142"). Sonuçlar RRF ile birleşir: `skor = Σ 1 / (k + sıra)`.
- **Reranker:** Vektör araması hızlı ama kabadır. Az sayıda aday LLM'e 0–10 arası puanlatılır, en iyiler kalır.
- **Guardrails:** Girdide prompt injection engellenir; kişisel veri `[MASKED_EMAIL]` gibi etiketlerle maskelenir.

Görev 1: Chunker splitter'ları (Kolay · 10 dk)

- **Dosya:** `ingestion/document_chunker.py` → `DocumentChunker.__init__`
- **Amaç:** Üç splitter'ı oluşturmak.

Yapılacaklar:

Alan	Sınıf	Parametreler
`self.header_splitter`	`MarkdownHeaderTextSplitter`	`headers_to_split_on=HEADERS`, `strip_headers=header_splitter_strip_headers`
`self.semantic_splitter`	`SemanticChunker`	`embeddings`, `breakpoint_threshold_type`, `breakpoint_threshold_amount`, `min_chunk_size=min_chunk_chars`, `buffer_size`
`self.size_splitter`	`RecursiveCharacterTextSplitter`	`chunk_size=max_chunk_chars`, `chunk_overlap=overlap_chars`, `is_separator_regex=True`, `separators=SENTENCE_SPLIT_RE`, `length_function=len`

İpucu: Parametre adları sınıfların kendi imzasıyla aynıdır; imzadaki değeri aynı adlı alana ver.

Doğrulama: `uv run pytest tests/exercises -k gorev1 -v`

Görev 2: Hibrit vector store (Orta · 10 dk)

- **Dosya:** `retrieval/vectordb_service.py` → `get_vector_store`
- **Amaç:** Dense ve sparse aramayı birlikte yapan Qdrant store'unu döndürmek.

Yapılacaklar:

- `QdrantVectorStore` oluştur ve döndür.
- `retrieval_mode=RetrievalMode.HYBRID` ver.
- `vector_name="dense"` ve `sparse_vector_name="sparse"` ver; adlar koleksiyon şemasıyla aynı olmalıdır.
- Servisin `self.client`, `self.embedding` ve `self.sparse_embedding` alanlarını kullan.

```
from langchain_qdrant import QdrantVectorStore, RetrievalMode

store = QdrantVectorStore(
    client=..., collection_name=..., embedding=..., sparse_embedding=...,
    retrieval_mode=RetrievalMode.HYBRID,
    vector_name="dense", sparse_vector_name="sparse",
)
```

Doğrulama: `uv run pytest tests/exercises -k gorev2 -v`

Görev 3: Reranker filtreleme ve sıralama (Orta · 20 dk)

- **Dosya:** `retrieval/reranking.py` → `rerank_documents`
- **Amaç:** LLM'in verdiği puanları güvenli şekilde filtreleyip sıralamak.
- **Hazır olan:** LLM çağrısı. Elinde `response.scores` vardır (her biri `RerankItem(id, score)`); `id`, `candidates` listesindeki indekstir.

Yapılacaklar:

1. `response.scores` listesini skora göre **azalan** sırala.
2. Şunları atla: `score < min_score`, daha önce görülmüş `id`, `id >= len(candidates)`.

3. `candidates[item.id]` belgesinin kopyasını, metadata'sına `"reranker_score"` ekleyerek al: `doc.model_copy(update={"metadata": {...}})`.
4. İlk `top_k` belgeyi döndür.

Dikkat:

- LLM aynı `id`'yi iki kez veya olmayan bir `id` döndürebilir; görülen `id`'leri bir `set`'te tut.
- Belgeyi yerinde değiştirme (`doc.metadata[...] = ...`); kopya al.
- Sıralamayı `top_k` kesmeden önce yap.
- Eşikteki skor kalmalıdır (`<`, `<=` değil).

Doğrulama: `uv run pytest tests/exercises -k gorev3 -v`

Görev 4: Input guardrail (Kolay · 10 dk)

- **Dosya:** `guardrails/guardrails.py` → `input_guardrail`
- **Amaç:** Injection'ı engellemek, diğer mesajlarda kişisel veriyi maskelemek.

Yapılacaklar:

1. `self._is_prompt_injection(message)` doğruysa uyarı logla ve `InputGuardrailResult(is_input_safe=False, message=None)` döndür.
2. Değilse `InputGuardrailResult(is_input_safe=True, message=self._mask_message(message))` döndür.

Dikkat:

- Injection durumunda maskeli mesaj döndürme; `message=None` olmalı.
- `_mask_message` maskelenecek bir şey yoksa `None` döner; olduğu gibi bırak.

Doğrulama: `uv run pytest tests/exercises -k gorev4 -v`

Bölüm sonu kontrolü: `uv run pytest tests/exercises/test_01_rag.py -v` → hepsi yeşil.

Bölüm 2 — LangGraph

Ön bilgi

- **State:** Graph boyunca taşınan sözlük (`AgentState`, `models/agent_state_model.py`). Önemli alanlar: `messages`, `intent`, `resolved_query`, `reranked_documents`, `generation`, `judge_passed`, `retry_count`.
- **Düğüm (node):** State'i okuyup yalnızca **değişen alanları** içeren bir sözlük döndüren async fonksiyon. `{"messages": [AIMessage(...)]}` döndürmek listeye ekler, üzerine yazmaz.
- **Fabrika kalıbı:** Düğüm yalnızca `state` alır; zincir ve servisler dış fonksiyondan gelir.
- **Structured output zinciri:** `TEMPLATE | llm.with_structured_output(Şema)` LLM cevabını bir Pydantic şemasına zorlar.
- **Router:** State'e bakıp bir sonraki düğümün **adını** döndüren sade fonksiyon; state'i değiştirmez.
- **`RetryPolicy`:** Geçici hatada (ör. API zaman aşımı) düğümü yeniden dener; LLM çağıran düğümlere verilir.

```
def make_greeting_node(prefix: str):                        # fabrika
    async def greeting_node(state: AgentState) -> dict:     # düğüm
        return {"generation": f"{prefix}, {state.get('user_name', 'misafir')}"}
    return greeting_node

builder = StateGraph(AgentState, context_schema=Context)
builder.add_node("intent", intent_node, retry_policy=RetryPolicy(max_attempts=3))
builder.add_edge(START, "intent")                           # sabit kenar
```

```
builder.add_conditional_edges("intent", my_router)        # router'ın döndürdüğü ada gider
graph = builder.compile(checkpointer=cp, store=store)
```

Görev 5: Intent zinciri ve düğümü (Orta · 20 dk)

- **Dosya:** `nodes/intent_node.py` → `build_intent_chain`, `make_intent_node`
- **Amaç:** Mesajın niyetini LLM ile belirleyen zinciri ve düğümü yazmak.

Yapılacaklar:

1. `build_intent_chain` içinde:
   - `build_chat_model(model_name, temperature=temperature)` ile LLM oluştur.
   - `INTENT_TEMPLATE | llm.with_structured_output(Intent)` döndür.
2. `make_intent_node` içinde:
   - Son kullanıcı mesajı: `get_user_last_message(state)`
   - Geçmiş: `trim_history(state, max_tokens=max_history_tokens)`
   - `response = await chain.ainvoke({"input_text": last_user_msg.text, "history": history})`
   - `response.confidence < min_confidence` ise niyeti `"needs_clarification"` yap.
   - Şunu döndür: `{"intent": ..., "intent_confidence": response.confidence, "resolved_query": response.resolved_query}`

İpucu: Eşit güven kabul edilir (`>=`); düşük güvenli tahmine güvenmek yerine netleştirme istenir.

Doğrulama: `uv run pytest tests/exercises -k gorev5 -v`

Görev 6: Router'lar (Kolay · 15 dk)

- **Dosya:** `graphs/routers.py`
- **Amaç:** Tablodaki kuralları dört router fonksiyonuna dökmek.

Tablo yukarıdan aşağı okunur; **ilk eşleşen kazanır**.

Router	Koşul	Dönüş
`route_after_input_guardrails`	`is_input_safe` yok veya `False`	`"blocked"`
	aksi halde	`"intent"`
`route_after_memory_recall`	`intent == "knowledge_question"`	`"retrieval"`
	aksi halde	`"assistant"`
`route_after_judge`	`judge_blocked`	`"blocked"`
	`judge_passed`	`"output_guardrails"`
	`retry_count >= max_retries` (yoksa 0)	`"output_guardrails"`
	aksi halde	`"assistant"`
`route_after_output_guardrails`	`len(messages) % batch_size == 0`	`"memory_extract"`
	aksi halde	`"__end__"`

İpucu: Anahtar state'te olmayabilir; `state.get(...)` kullan.

Dikkat: Judge'da sıra önemlidir; `judge_blocked`, `judge_passed`'ten önce kontrol edilir. Sınır `>=`'dir.

Doğrulama: `uv run pytest tests/exercises -k gorev6 -v`

Görev 7: Workflow graph (Zor · 30 dk)

- **Dosya:** `graphs/graph.py` → `workflow_graph`
- **Amaç:** Tüm düğümleri ve kenarları bağlayıp graph'ı derlemek.
- **Hazır olan:** Zincirlerin kurulumu.

Yapılacaklar:

1. **Kâğıt üzerinde:** Bölüm 5'teki akışı ve Görev 6 tablosunu kullanarak düğümleri kutu, geçişleri ok olarak çiz; koşullu okların üzerine koşulu yaz.
2. **Düğümleri ekle** (adları aynen kullan):

Düğüm adı	Fabrika
`input_guardrails`	`make_input_guardrails_node(guardrails_service)`
`intent`	`make_intent_node(intent_chain)`
`memory_recall`	`make_memory_recall_node()`
`retrieval`	`make_retrieval_node(retrieval_service)`
`reranker`	`make_reranker_node(reranker_chain)`
`assistant`	`make_assistant_node(assistant_chain)`
`judge`	`make_judge_node(judge_chain)`
`output_guardrails`	`make_output_guardrails_node(guardrails_service)`
`memory_extract`	`make_memory_extract_node(memory_chain)`
`blocked`	`blocked_response_node`

3. **Kenarları ekle:** Sabit kenarlar için `add_edge`, router'lı düğümler için `add_conditional_edges`.
4. **Derle:** `return builder.compile(checkpointer=checkpointer, store=store)`

İpuçları:

- LLM çağıran düğümlere (`intent`, `retrieval`, `reranker`, `assistant`, `judge`) `retry_policy=RetryPolicy(max_attempts=settings.LLM_MAX_ATTEMPTS)` ver.
- Router'ı olan bir düğümden ayrıca `add_edge` yazma.
- `blocked` ve `memory_extract` doğrudan `END`'e gider.
- Router'ın döndürdüğü ad ile `add_node` adı birebir aynı olmalıdır.

Doğrulama: `uv run pytest tests/exercises -k gorev7 -v`

Bölüm sonu kontrolü: `uv run pytest tests/exercises/test_02_langgraph.py -v` → hepsi yeşil.

Bölüm 3 — Tool ve Agent

Ön bilgi

- **Tool (araç):** LLM'in çağırabileceği fonksiyon. `@tool` fonksiyonu araca çevirir; **docstring**, LLM'e aracın ne zaman ve nasıl kullanılacağını anlatır. Hazır örnek: `tools/date_tool.py`.
- **ToolRuntime:** Araca, LLM'in görmediği çalışma zamanı bilgisini verir. `runtime.context`, agent çağrılırken verilen `context=` nesnesidir. Bu projede `AgentContext.sources`, bir turda bulunan tüm kaynakları biriktirir.

- **Agent:** Model → araç çağrısı → araç sonucu → model döngüsü; model cevap verene kadar sürer. `create_agent` bu döngüyü hazır kurar.
- **Middleware:** Döngüye kural ekler. `ToolCallLimitMiddleware` bir aracın çağrı sayısını sınırlar; `ToolRetryMiddleware` hata veren aracı yeniden dener, olmazsa `on_failure` metnini modele verir.
- **ToolStrategy:** Agent'ın son cevabını bir şemaya (`AssistantResponse`) uygun, yapılandırılmış olarak vermesini sağlar.

```
@tool
async def search_documents(query: str, runtime: ToolRuntime[AgentContext]) -> str:
    """Yüklenen dokümanlarda arama yapar."""
    runtime.context.sources.extend(...)
```

Görev 8: `search_documents` aracı (Orta · 20 dk)

- **Dosya:** `tools/retrieve_tool.py`
- **Amaç:** Aday getirip rerank eden ve bulunan kaynakları numaralandırarak biriktiren aracı tamamlamak.
- **Hazır olan:** Boş sorgu kontrolü.

Yapılacaklar — aday getirme ve rerank:

1. `candidate_docs = await retrieval_service.search(query=query, top_k=candidates)` (burada `top_k` değil, geniş havuz olan `candidates`).
2. Aday yoksa `NOT_FOUND_ANSWER` döndür.
3. `rerank_documents(chain=reranker_chain, query=query, candidates=candidate_docs, top_k=top_k, min_score=min_score)` çağır. `RerankerError` olursa uyarı logla ve `candidate_docs[:top_k]` ile devam et.
4. Belge kalmadıysa `NOT_FOUND_ANSWER` döndür.

Yapılacaklar — kaynak biriktirme:

1. `start = len(runtime.context.sources) + 1`
2. `runtime.context.sources.extend(documents)`
3. `return f"Bulunan kaynaklar:\n\n{format_sources(documents, start=start)}"`

Neden `start`? Asistan bir turda aracı iki kez çağırabilir: ilk arama `[1][2]`, ikinci arama `[3][4]` döndürmelidir. Aksi halde `[1]` atfı yanlış belgeyi gösterir.

Dikkat:

- `start`'ı `extend`'den **önce** hesapla.
- `runtime.context.sources = documents` yazma; önceki kaynaklar silinir.
- Yalnızca `RerankerError` yakala; `except Exception` kullanma.

Doğrulama: `uv run pytest tests/exercises -k gorev8 -v`

Görev 9: Agent kurulumu (Zor · 20 dk)

- **Dosya:** `nodes/assistant_node.py` → `build_assistant_agent`
- **Amaç:** Araçları, middleware'leri ve çıktı şemasıyla agent'ı oluşturmak. Değerler fonksiyonun docstring'indedir.

Yapılacaklar:

1. `llm = build_chat_model(model_name, temperature=temperature)`
2. Her araç için **ayrı** bir `ToolCallLimitMiddleware` yaz (`SEARCH_TOOL_NAME` ve `DATE_TOOL_NAME`; `run_limit=settings.MAX_TOOL_CALLS`).
3. `ToolRetryMiddleware(max_retries=..., on_failure=_search_tool_failure_message)` ekle.
4. `create_agent(...)` döndür:
    - `tools=[make_search_documents_tool(retrieval_service, reranker_chain), get_today_date]`

- `response_format=ToolStrategy(AssistantResponse)`
- `context_schema=AgentContext`
- `middleware=` yukarıdaki liste

Doğrulama: `uv run pytest tests/exercises -k gorev9 -v`

Görev 10: Agent düğümü ve agentic graph (Orta · 10 dk)

- **Dosyalar:** `nodes/assistant_node.py` → `make_agent_assistant_node`; `graphs/graph.py` → `agentic_graph`
- **Amaç:** Agent'ı graph'a düğüm olarak bağlamak.

Yapılacaklar — `make_agent_assistant_node`:

1. `messages = AGENT_ASSISTANT_TEMPLATE.format_messages(memories=_format_memories(state), intent=state["intent"], resolved_query=state.get("resolved_query") or last_user_msg.text, feedback=format_feedback(state.get("feedback")), history=history)`
2. `agent_context = AgentContext()` — `try`'dan **önce**, her çağrıda yeni.
3. `result = await agent.ainvoke({"messages": messages}, context=agent_context)`
4. En sonda `return _assistant_update(response, agent_context.sources)`

Yapılacaklar — `agentic_graph` (workflow graph'tan farkları):

- `retrieval` ve `reranker` düğümleri **yoktur**.
- Asistan düğümü `make_agent_assistant_node(assistant_agent)` ile kurulur.
- `memory_recall → assistant` sabit kenardır; `route_after_memory_recall` kullanılmaz.

Not: Agent'ta kaynaklar state'ten değil, aracın `agent_context.sources`'a biriktirdiklerinden gelir.

Doğrulama: `uv run pytest tests/exercises -k gorev10 -v`

Bölüm sonu kontrolü: `uv run pytest tests/exercises -q` → hepsi yeşil.

---

6. Uçtan uca çalıştırma

1. Uygulamayı başlat (ilk derleme birkaç dakika sürer):

   ```
   docker compose up --build
   ```

2. Tarayıcıda http://localhost:7860 adresini aç.
3. Bir PDF yükle (chunklama biraz sürebilir) ve bir soru sor.
4. Graph türünü değiştirip **aynı soruyu** iki modda da sor.
5. Terminal loglarında farkı gör: workflow'da `retrieval` düğümü, agentic'te `search_documents` aracı çağrılır.
6. Durdurmak için `Ctrl+C`. Kodu değiştirirsen komutu tekrar çalıştır.

7. Kontrol listesi

- ☐ 10 görevin testleri yeşil (`uv run pytest tests/exercises -q`)
- ☐ Uygulama Docker ile açıldı
- ☐ Aynı soru workflow ve agentic modda denendi
- ☐ Aşağıdaki sorular cevaplandı

8. Sorular

1. Hibrit aramada neden hem dense hem BM25 kullanılır? Hangisi hangi sorguda daha iyidir?
2. Judge router'ında `retry_count >= max_retries` koşulu olmasaydı ne olurdu?
3. Aynı soruyu workflow ve agentic graph'a sorduğunda loglarda ne fark gördün? Hangisini ne zaman seçerdin?